Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ

КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

РУБЕЖНЫЙ КОНТРОЛЬ №1

ДИСЦИПЛИНА: «Парадигмы и конструкции языков программирования»

Предметная область № 2: Школьник — Класс
Вариант запросов А

Выполнил: _____ Бабичев А.А.
(Подпись) (Ф.И.О.)

Проверил: _____ Гапанюк Ю.Е.
(Подпись) (Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:

- Оценка:

Постановка задачи

Реализованы связи один-ко-многим (основной класс школьника) и многие-ко-многим (посещение учебных классов). Количественный признак школьника — балл за контрольную работу. Для обоих типов связей созданы согласованные по идентификаторам тестовые данные.

Запрос A1 выводит школьников и их основные классы с сортировкой по названию класса. Запрос A2 считает сумму баллов школьников в каждом основном классе и сортирует суммы по убыванию. Запрос A3 выводит названия со словом «класс» и всех посещающих их школьников по связи многие-ко-многим.

Текст программы

```
# -*- coding: utf-8 -*-
"""РК №1: запросы A, предметная область №2 «Школьник — Класс»."""

from operator import itemgetter


class Schoolchild:
    """Школьник: сторона «много» в отношении один-ко-многим."""

    def __init__(self, id, surname, score, class_id):
        self.id = id
        self.surname = surname
        self.score = score  # Балл за контрольную работу
        self.class_id = class_id


class SchoolClass:
    """Класс: сторона «один» в отношении один-ко-многим."""

    def __init__(self, id, name):
        self.id = id
        self.name = name


class SchoolchildClass:
    """Связь школьников с посещаемыми ими учебными классами."""

    def __init__(self, schoolchild_id, class_id):
        self.schoolchild_id = schoolchild_id
        self.class_id = class_id


classes = [
    SchoolClass(1, "Класс 5А"),
    SchoolClass(2, "Класс 5Б"),
    SchoolClass(3, "Математический класс 5В"),
    SchoolClass(4, "5Г"),
]

schoolchildren = [
    Schoolchild(1, "Антонов", 78, 1),
    Schoolchild(2, "Иванова", 85, 1),
    Schoolchild(3, "Петров", 74, 2),
    Schoolchild(4, "Сидоров", 92, 3),
    Schoolchild(5, "Андреева", 88, 4),
]

schoolchildren_classes = [
    SchoolchildClass(1, 1), SchoolchildClass(1, 3),
    SchoolchildClass(2, 1), SchoolchildClass(2, 2),
    SchoolchildClass(3, 2), SchoolchildClass(3, 4),
    SchoolchildClass(4, 3), SchoolchildClass(4, 1),
    SchoolchildClass(5, 4), SchoolchildClass(5, 3),
```

```
]


def main():
    # Один-ко-многим: основной класс каждого школьника.
    one_to_many = [
        (child.surname, child.score, school_class.name)
        for school_class in classes
        for child in schoolchildren
        if child.class_id == school_class.id
    ]

    # Многие-ко-многим: посещаемые учебные классы.
    many_to_many = [
        (child.surname, school_class.name)
        for link in schoolchildren_classes
        for child in schoolchildren
        for school_class in classes
        if child.id == link.schoolchild_id and school_class.id == link.class_id
    ]

    # A1. Связанные школьники и классы, сортировка по названию класса.
    result_a1 = sorted(one_to_many, key=itemgetter(2, 0))
    print("Задание A1. Школьники и их основные классы")
    for surname, score, class_name in result_a1:
        print(f"{class_name}: {surname} (балл: {score})")

    # A2. Сумма баллов в каждом основном классе, по убыванию суммы.
    result_a2 = sorted(
        [
            (
                school_class.name,
                sum(score for _, score, name in one_to_many
                    if name == school_class.name),
            )
            for school_class in classes
            if any(name == school_class.name for _, _, name in one_to_many)
        ],
        key=itemgetter(1),
        reverse=True,
    )
    print("\nЗадание A2. Сумма баллов по основным классам")
    for class_name, total in result_a2:
        print(f"{class_name}: {total}")

    # A3. Названия со словом «класс» и посещающие их школьники.
    result_a3 = {
        school_class.name: [
            surname for surname, name in many_to_many
            if name == school_class.name
        ]
        for school_class in classes
        if "класс" in school_class.name.lower()
    }
    print("\nЗадание A3. Учебные классы со словом «класс»")
    for class_name, surnames in result_a3.items():
        print(f"{class_name}: {', '.join(surnames)}")


if __name__ == "__main__":
    main()
```

Результат выполнения

```
Задание A1. Школьники и их основные классы
5Г: Андреева (балл: 88)
Класс 5А: Антонов (балл: 78)
Класс 5А: Иванова (балл: 85)
Класс 5Б: Петров (балл: 74)
Математический класс 5В: Сидоров (балл: 92)

Задание A2. Сумма баллов по основным классам
Класс 5А: 163
Математический класс 5В: 92
5Г: 88
Класс 5Б: 74

Задание A3. Учебные классы со словом «класс»
Класс 5А: Антонов, Иванова, Сидоров
Класс 5Б: Иванова, Петров
Математический класс 5В: Антонов, Сидоров, Андреева
```